

CUDA PROGRAMMING FOR NVIDIA H100S

Stream-K



Prateek Shukla · 简体中文版

第一性原理

对一个输出 tile C_{tile} , GEMM 计算 :

$$C_{\text{tile}} = \text{sum over K-tiles of } (A_{\text{tile}_k} * B_{\text{tile}_k})$$

如果一个 CTA 为该输出 tile 计算全部 K-tile , 就不需要跨 CTA 归约。

如果多个 CTA 各自只计算 K-tile 的一个子集 , 每个 CTA 产出一个部分和 , 而这些部分和必须被合并。

在这个调度器中 , 那个合并步骤被称为 fixup。

工作单元

每个工作单元包括：

- tile 坐标 (m_idx, n_idx, l_idx)
- k_idx : 输出 tile 内的起始 k-tile
- k_tile_count : 该工作单元为这个输出 tile 计算的 k-tile 数量

是否需要归约由以下决定：

- k 方向完整 tile : $k_tile_count == k_tiles_per_output_tile$ -> 不需要归约
- k 方向部分 tile : $k_tile_count \neq k_tiles_per_output_tile$ -> 需要归约

这就是 `requires_fixup(...)` 背后的关键谓词。

Split

一个 CTA 被分配的 k-tile 迭代区间可能落在一个输出 tile 的中间。例如，3 个输出 tile 各有 90 个 k-tile，而有 4 个 CTA 单元：

一个"split"是一个 CTA 对单个输出 tile 的贡献。上面的 Unit 1 有两个 split——一个对应 tile 0 的尾部，一个对应 tile 1 的头部。代码逐个处理它们（这就是 `advance_to_next_work` 中的 `k_tile_remaining` 循环）。

跟踪三元组

对一个单独的 split (一个 CTA 在一个输出 tile 上的工作) :

K_idx : 该 split 在输出 tile 的 K 维度内部从哪里开始。

$K_idx = tile_iter_start - output_tile_iter_start$ 。

对 Unit 0 在 tile 0 上的工作 , $K_idx = 0$ 。对 Unit 1 在 tile 0 上的工作 , $K_idx = 67$ 。

k_tile_count : 该 split 处理多少个 k-tile。对 tile 0 上的 Unit 0 , 是 67。对 tile 0 上的 Unit 1 , 是 23 (= $90 - 67$) 。

$is_final_split()$: $(K_idx + k_tile_count) == k_tiles_per_output_tile$ 。当该 split 覆盖 K 维度的末尾时为真。Unit 1 在 tile 0 上的 split 是一个 final split ($67 + 23 = 90$) 。

三种角色直接得出

给定一个输出 tile 可能有 2-4 个 CTA 各自计算 K 的一部分：

$K_idx == 0 \rightarrow$ 你是第一个 split。你计算了 k-tile $[0, N)$ 。在你之前没有人向工作区写过任何东西。直接存储即可。

$is_final_split() == true$ 且不是 separate reduction $\rightarrow$ 你是 final split 和 epilogue 所有者。你计算了 k-tile $[X, 90)$ 。等待你之前的所有人，加载它们的累加结果，加上你的，运行 epilogue。

其余一切 $\rightarrow$ 你是中间 split。你计算了 k-tile $[A, B)$ ，其中 $0 < A < B < 90$ 。你需要把自己的部分和归约进工作区里已有的内容。

锁：它在物理上是什么

全局内存中有一个连续的 `int` 数组，每个输出 tile 一个（多 warpgroup kernel 还要乘以 `num_barriers`）。指向这个数组的指针紧挨着归约数据缓冲区，位于同一次分配之中。kernel 启动时，每把锁都从 0 开始。锁是一个单独的整数，编码了针对给定输出 tile 已完成并写入工作区的 K 维度工作量。在普通（非 `separate reduction`）模式下，它统计已处理的累计 k-tile 数。它只会增加。锁以 K-tile 空间编码进度。对确定性模式，每个 `split` 等待与其起始位置匹配的精确累计 K-tile 数，从而强制一个严格的从左到右归约顺序。对非确定性模式，中间 `split` 只需要知道工作区已被初始化（`lock >= 1`），然后它们就竞相以原子方式往里归约。

分组

分组是一种 L2 缓存局部性优化。它们把 stream-K 单元划分成 G 个独立子组，每个组只在自己的 stream-K tile 子集上协作。这是 stream-K 特有的优化，因为 stream-K 破坏了那种漂亮的、基于波的光栅化模式——单个 CTA 可能横跨来自输出 grid 不同区域的 tile，从而摧毁那种局部性。没有分组 ($G=1$) 时，所有 stream-K 单元共享一个大池子，横跨所有 stream-K 输出 tile 的 K tile。Unit 0 可能在做 tile 0 和 tile 1，而 unit 7 在做 tile 5 和 tile 6——完全不同的空间位置。它们在 L2 缓存中的数据毫无重叠。有了分组之后，每个组里的 unit 0 会计算相同 K 范围的 tile——按照数据并行表述的光栅化顺序，这些 tile 本会被分配到同一条波里

分组层级

分组（最多 8 个，为了 L2 局部性）

每个组包含多个 cluster-tile

每个集群（cluster）包含多个 CTA（线程块）

每个 CTA 处理 K-tile

分组沿光栅化维度确定。例如，如果你沿 M 光栅化，且 $\text{problem_blocks_m} / \text{cluster_m} = 4$ ，你会得到 4 个组。各组在输出空间中交错（interleave）。最终的 `output_tile_id` 计算如下：
$$\text{output_tile_id} = (\text{output_tile_id_in_group} * \text{num_groups}) + \text{group_idx}$$

HyTiS

HyTiS 通过让部分波 (partial wave) 使用更细粒度的 tile 来解决波量化, 让更多 SM 保持忙碌。它是一种纯空间分解 ($M \times N$)、使用异构 tile 大小, 对比 Stream-K 沿 K 维度分解、使用同构 tile 大小。HyTiS 不用 stream-k, 而是在一次 kernel launch 中使用两种不同的 tile 大小:

- 大 tile (例如 128×256) 给完整的波 -> 最大吞吐量
- 小 tile (例如 64×64) 给部分波 -> 最小延迟

没有归约、没有工作区、没有屏障、没有 fixup 代价是: 当问题在 M 和 N 上很小但在 K 上很大时, HyTiS 帮不上忙